Invcstigación: Fundamcntos y Arquitcctura dc los Sistcmas dc Bascs dc Datos

Instituto Politécnico Nacional · Escuela Superior de Cómputo (ESCOM)

Materia: Bases de Datos · Profesor: Gabriel Hurtado Aviles · Grupo: 3CV4

Práctica 1: "Modelo Entidad Relación" ·

Ejercicio 3: Investigación, qué es una base de datos

Integrante: Jiménez Sánchez Taylin

Introducción

Este trabajo reúne una investigación sobre los contenidos de la Unidad Temática I del curso, apoyada en libros de texto de referencia y en artículos científicos arbitrados. El propósito es entender qué es una base de datos, quiénes intervienen en ella, cómo se desarrolla y de qué componentes se compone un sistema gestor, de modo que sea más sencillo pasar de la teoría al modelado conceptual y al diseño de soluciones de software. Al final se incluye una conclusión personal y la lista de referencias en formato APA.

1. Dato, información y base de datos

Un **dato** es una representación simbólica, numérica o textual de un hecho concreto que, por sí sola, carece de contexto. El número "38" es un dato: no indica si se trata de una edad, una temperatura o un precio. Cuando ese dato se procesa, se organiza y se coloca en un contexto (por ejemplo, "38 °C es la temperatura corporal de un paciente"), se convierte en **información**, es decir, en algo que tiene significado y puede apoyar una decisión (Coronel & Morris, 2019).

En cuanto al concepto de **base de datos**, distintos autores ofrecen definiciones que se complementan:

- **Silberschatz, Korth y Sudarshan (2019)** la presentan como un conjunto de datos interrelacionados junto con los programas que permiten consultarlos y modificarlos. Su definición pone el acento en la parte operativa: los datos solo son útiles si existe software que los gestione de forma eficiente.
- **Elmasri y Navathe (2016)** la describen como una colección de datos relacionados con un significado implícito, y añaden la idea de *minimundo*: la base de datos modela solo la porción de la realidad que le interesa a una organización.

- **Connolly y Begg (2015)** la definen como un conjunto compartido de datos lógicamente relacionados, junto con su descripción, diseñado para cubrir las necesidades de información de una organización.

Comparación. Silberschatz et al. destacan el aspecto sistémico (almacenar y manipular con ayuda de programas), Elmasri y Navathe privilegian el aspecto semántico (qué parte del mundo real representan los datos) y Connolly y Begg subrayan el carácter compartido y la existencia de una descripción de los datos. Las tres miradas son compatibles: una base de datos es, a la vez, un modelo de una parte de la realidad y una infraestructura de software para usarlo.

2. Características fundamentales

Un sistema de base de datos se distingue de otras formas de almacenamiento por las siguientes propiedades:

- **Integración.** Los datos se ven como una sola estructura lógica, lo que evita "islas" de información aisladas entre áreas de una organización.
- **Persistencia.** La información sobrevive a la ejecución de los programas porque se guarda en medios no volátiles.
- **Redundancia controlada.** El sistema reduce la duplicación de datos; cuando se duplica a propósito (por ejemplo, para mejorar el rendimiento), el propio gestor se encarga de mantener la coherencia (Elmasri & Navathe, 2016).
- **Integridad.** Garantiza que los datos sean correctos y válidos. Se manifiesta en tres formas: integridad de entidad (claves primarias no nulas ni repetidas), integridad referencial (las claves foráneas apuntan a registros existentes) e integridad de dominio (los valores respetan los tipos y rangos permitidos).
- **Independencia de los datos.** Permite cambiar la organización física o lógica de los datos sin tener que reescribir las aplicaciones que los usan.
- **Seguridad y privacidad.** Mediante autenticación y autorización se controla quién puede ver o modificar cada dato.
- **Concurrencia.** Varios usuarios pueden trabajar al mismo tiempo sin interferirse; esto se logra con mecanismos de control como los bloqueos (*locks*) que aseguran el aislamiento entre transacciones.
- **Recuperación.** Tras una falla, el sistema puede volver a un estado consistente gracias al registro de transacciones (*log*) y a los puntos de control (*checkpoints*) (Haerder & Reuter, 1983).

3. Sistemas de archivos frente a bases de datos

Antes de que se generalizaran los sistemas gestores, cada aplicación de una organización administraba sus propios archivos. Ese enfoque tiene limitaciones importantes:

1. **Redundancia e inconsistencia.** Un mismo dato aparece en varios archivos y formatos, y actualizarlo en uno no lo actualiza en los demás.
2. **Dependencia entre programas y datos.** Si cambia el formato de un registro, hay que modificar todos los programas que leen ese archivo.
3. **Poca flexibilidad.** Responder una consulta no prevista o generar un reporte nuevo exige escribir más código.
4. **Acceso concurrente deficiente.** Los archivos no ofrecen un control fino para que dos personas modifiquen el mismo registro sin perder cambios.
5. **Aislamiento de los datos y seguridad limitada.** Los datos quedan dispersos en formatos distintos y es difícil aplicar reglas de acceso uniformes (Silberschatz et al., 2019).

Estas deficiencias impulsaron la aparición de los **sistemas gestores de bases de datos (SGBD)**, que funcionan como una capa de abstracción: centralizan la administración de los datos y separan su definición de su uso.

4. Usuarios y roles

En torno a una base de datos participan varios perfiles con responsabilidades distintas:

Rol	Responsabilidades principales	Privilegios típicos
Administrador de la base de datos (DBA)	Seguridad, rendimiento, respaldos y recuperación	Acceso total (superusuario)
Diseñador de la base de datos	Definir estructuras de datos y restricciones de integridad	Diseño conceptual y lógico
Programador de aplicaciones	Construir interfaces y lógica de negocio que usan la base de datos	Ejecución de sentencias DML
Usuario avanzado	Realizar consultas ad hoc y análisis con SQL u otras herramientas	Consulta y análisis de datos
Usuario ingenuo (*naïve*)	Operar mediante aplicaciones o formularios ya construidos	Acceso limitado a la interfaz final

5. Ciclo de vida del desarrollo de una base de datos

Connolly y Begg (2015) proponen un proceso sistemático dividido en etapas, cada una con un producto o entregable asociado:

Etapa	Descripción	Entregable
1. Planificación	Definir el alcance, los recursos y los plazos	Plan del proyecto
2. Definición del sistema	Delimitar usuarios y áreas de aplicación	Documento de requisitos
3. Recolección y análisis de requisitos	Identificar las necesidades de datos de la organización	Especificación funcional
4. Diseño de la base de datos	Conceptual (independiente del software), lógico (mapeo al modelo relacional) y físico (estructuras de almacenamiento)	Diagrama Entidad-Relación, esquema relacional y diccionario de datos
5. Selección del SGBD	Evaluar productos según las necesidades del proyecto	Informe de selección

Etapa	Descripción	Entregable
6. Diseño de aplicaciones	Diseñar la interfaz y los programas que usarán la base de datos	Prototipos de interfaz
7. Implementación	Crear las definiciones de la base de datos y los programas	Código SQL y aplicaciones
8. Conversión y carga de datos	Migrar los datos existentes al nuevo sistema	Base de datos poblada
9. Pruebas	Detectar errores y medir el desempeño	Plan y resultados de pruebas
10. Mantenimiento y evolución	Corregir y adaptar el sistema tras ponerlo en operación	Bitácora de cambios

El diseño conceptual suele apoyarse en el modelo Entidad-Relación propuesto por Chen (1976), que representa la realidad mediante entidades, atributos y relaciones, y es el punto de partida de esta práctica.

6. Sistema gestor de bases de datos (SGBD)

Un SGBD es el software que permite crear, mantener y utilizar una base de datos. Hellerstein, Stonebraker y Hamilton (2007) describen su arquitectura en varios componentes; de forma simplificada, se agrupan en dos grandes bloques:

- **Procesador de consultas.** Incluye el *parser* (analiza la sintaxis de la consulta), el optimizador (elige el plan de ejecución más eficiente) y el motor de ejecución (lo ejecuta).
- **Gestor de almacenamiento.** Reúne el gestor de *buffer* (administra la memoria principal), el gestor de transacciones (garantiza la atomicidad y la recuperación), el gestor de bloqueos (controla la concurrencia) y el gestor de archivos (se comunica con el sistema operativo).

Sublenguajes de SQL

Sublenguaje	Función	Ejemplos
DDL (definición de datos)	Define y modifica la estructura	CREATE TABLE, ALTER, DROP
DML (manipulación de datos)	Consulta y modifica los datos	SELECT, INSERT, UPDATE, DELETE
DCL (control de datos)	Administra permisos de acceso	GRANT, REVOKE
TCL (control de transacciones)	Confirma o deshace transacciones	COMMIT, ROLLBACK

Clasificación de los SGBD

Los sistemas gestores pueden clasificarse según el modelo de datos (relacional, NoSQL, entre otros), el número de usuarios que soportan (monousuario o multiusuario), su distribución (centralizados o distribuidos) y su modelo de licenciamiento (propietarios o de código abierto) (Elmasri & Navathe, 2016).

7. Arquitectura y entorno del sistema

La arquitectura **ANSI/SPARC** organiza el sistema en tres niveles de abstracción con el fin de lograr independencia de los datos (Tsichritzis & Klug, 1978):

1. **Nivel externo (vistas).** Muestra a cada grupo de usuarios únicamente la parte de los datos que le interesa.
2. **Nivel conceptual.** Describe qué datos se almacenan y cómo se relacionan para toda la organización.
3. **Nivel interno (físico).** Describe cómo se guardan realmente los datos en los dispositivos de almacenamiento.

Esta separación hace posible la **independencia lógica** (modificar el esquema conceptual sin afectar las vistas de los usuarios) y la **independencia física** (cambiar la forma de almacenamiento sin alterar el esquema conceptual).

Por su topología, un sistema puede ser **centralizado** (todo reside en una sola máquina) o de tipo **cliente-servidor**. En este último caso puede organizarse en dos capas (cliente y servidor de base de datos) o en *n* capas, cuando se agregan servidores de aplicaciones y servidores web entre el usuario y los datos.

8. Modelos de datos

Un modelo de datos es un conjunto de herramientas conceptuales para describir los datos, sus relaciones, su significado y sus restricciones. Sirve de puente entre la realidad que se quiere representar y su implementación en un sistema informático.

Por nivel de abstracción

- *Conceptual:* alto nivel, cercano a la forma en que los usuarios entienden el problema (por ejemplo, el modelo Entidad-Relación).
- *Lógico:* representa el diseño en términos de un modelo implementable (por ejemplo, tablas relacionales).
- *Físico:* bajo nivel, describe archivos, índices y estructuras de almacenamiento.

Por estructura

- *Relacional:* organiza los datos en relaciones (tablas) y se fundamenta en el álgebra relacional (Codd, 1970).
- *Jerárquico y de red:* modelos tempranos basados en árboles y grafos de registros.
- *Orientado a objetos:* combina conceptos de la programación orientada a objetos con persistencia de datos.
- *NoSQL y multimodelo:* pensados para escalar horizontalmente y manejar datos poco estructurados mediante esquemas clave-valor, de documentos, de columnas o de grafos (Cattell, 2011).

Conclusión

Jimenez Sanchez Taylin:

Con esta investigación pude afianzar los conceptos centrales de la Unidad Temática I. Entendí por qué los sistemas de archivos tradicionales resultaron insuficientes y cómo los sistemas gestores de bases de datos resolvieron problemas como la redundancia, la inconsistencia y la dependencia entre programas y datos, gracias a propiedades como la integridad, la concurrencia y la independencia de los datos.

También revisé la arquitectura de tres niveles, los distintos roles de usuario y las etapas del ciclo de vida de una base de datos, lo que me permite ubicar el diseño conceptual con el modelo Entidad-Relación dentro de un proceso más amplio. Por último, trabajar con literatura especializada y citarla en formato APA me ayudó a sustentar cada afirmación y a tener una visión más completa de cómo se modela, administra y preserva la información en los sistemas actuales.

Referencias

- Cattell, R. (2011). Scalable SQL and NoSQL data stores. *ACM SIGMOD Record, 39*(4), 12–27. https://doi.org/10.1145/1978915.1978919
- Chen, P. P. (1976). The entity-relationship model: Toward a unified view of data. *ACM Transactions on Database Systems, 1*(1), 9–36. https://doi.org/10.1145/320434.320440
- Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM, 13*(6), 377–387. https://doi.org/10.1145/362384.362685
- Connolly, T., & Begg, C. (2015). *Database systems: A practical approach to design, implementation, and management* (6th ed.). Pearson.
- Coronel, C., & Morris, S. (2019). *Database systems: Design, implementation, & management* (13th ed.). Cengage Learning.
- Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.). Pearson.
- Haerder, T., & Reuter, A. (1983). Principles of transaction-oriented database recovery. *ACM Computing Surveys, 15*(4), 287–317. https://doi.org/10.1145/289.291
- Hellerstein, J. M., Stonebraker, M., & Hamilton, J. (2007). Architecture of a database system. *Foundations and Trends in Databases, 1*(2), 141–259. https://doi.org/10.1561/1900000002
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database system concepts* (7th ed.). McGraw-Hill Education.
- Tsichritzis, D., & Klug, A. (1978). The ANSI/X3/SPARC DBMS framework: Report of the study group on database management systems. *Information Systems, 3*(3), 173–191.